

ESPECIALIZACION EN INTELIGENCIA ARTIFICIAL

Computacion en la Nube

INFORME MICROPROYECTO 2

Implementacion de Kubernetes en Azure (AKS)

Estudiante:

Milton Giovanni Jaramillo Herrera

Docente:

Prof. Heberth Fabian Martinez Velasquez

Universidad Autonoma de Occidente — UAO Virtual

Junio 2026

1. Introducción

La computación en la nube ha transformado la manera en que las organizaciones despliegan y gestionan sus aplicaciones. Los contenedores se han consolidado como la unidad básica de empaquetado de software, y Kubernetes (K8s) se ha convertido en el estándar de orquestación de contenedores a nivel industrial. A diferencia de las máquinas virtuales que virtualizan hardware completo, los contenedores comparten el kernel del sistema operativo anfitrión, lo que los hace más ligeros, portátiles y rápidos de iniciar.

Azure Kubernetes Service (AKS) es el servicio gestionado de Kubernetes de Microsoft Azure. Abstrae la complejidad del plano de control (API Server, etcd, Scheduler) haciéndolo responsabilidad de Azure sin costo adicional, permitiendo al usuario enfocarse en desplegar aplicaciones. En este microproyecto se implementó un cluster AKS desde cero usando una suscripción Azure for Students, se desplegaron aplicaciones de deep learning e infraestructura de monitoreo, y se exploró la supervisión integrada con Azure Monitor.

2. Objetivos

2.1 Objetivo General

Implementar un cluster AKS funcional y desplegar sobre él aplicaciones prácticas de inteligencia artificial y monitoreo, evaluando su comportamiento mediante las herramientas de supervisión de Azure.

2.2 Objetivos Específicos

- Actividad 1: Crear y verificar un cluster AKS de dos nodos desde Azure Portal y Azure CLI.
- Actividad 2: Desplegar un servidor Jupyter con TensorFlow y ejecutar un modelo de clasificación de imágenes MobileNetV2.
- Actividad 3: Desplegar Grafana como aplicación de interés y acceder a su dashboard públicamente.
- Actividad 4: Supervisar el cluster en tiempo real con Azure Monitor analizando métricas de CPU, memoria y pods.

3. Herramientas y Tecnologías Utilizadas

- Azure Kubernetes Service (AKS): servicio gestionado de Kubernetes en Microsoft Azure que abstrae el plano de control del cluster.
- kubectl: herramienta de línea de comandos oficial de Kubernetes para gestionar recursos del cluster.
- Azure CLI v2.87.0: interfaz de línea de comandos de Azure para autenticación y gestión de recursos desde Windows.
- Azure Cloud Shell: terminal bash integrada en el portal de Azure sin necesidad de instalación local.
- TensorFlow / Keras (imagen tensorflow/tensorflow:latest-jupyter): framework de deep learning con servidor Jupyter Notebook incluido.
- MobileNetV2: arquitectura de red neuronal convolucional preentrenada en ImageNet (1000 clases) optimizada para hardware limitado.
- Grafana v13.0.2: plataforma de código abierto para visualización de métricas y creación de dashboards.

- Azure Monitor: servicio de observabilidad nativo de Azure integrado con AKS para metricas de nodos, pods y plano de control.
- Standard_D2s_v3: tamaño de VM con 2 vCPU y 8 GiB RAM utilizado como nodo trabajador del cluster.

4. Descripcion del Proceso

4.1 Implementacion y Verificacion del Cluster AKS

Se inicio sesion en Azure Portal con la cuenta institucional UAO activada en Azure for Students (\$100 USD de credito). Se creo el cluster aks-microproyecto2 en el grupo de recursos rg-microproyecto2, region East US 2, con Kubernetes 1.34.8, plan Gratis y 2 nodos Standard_D2s_v3 con escalado manual.

Dificultades encontradas: la Serie B de VMs no es compatible con AKS por no admitir virtualizacion anidada; Standard_DS2_v2 no estaba disponible en East US 2 para cuentas de estudiante; la cuota de 4 vCPU de Azure for Students obliga a usar exactamente 2 nodos (2 x 2 vCPU). La implementacion completo en aproximadamente 8 minutos.

Se verifico el cluster desde Azure Cloud Shell con los siguientes comandos:

```
az aks get-credentials --resource-group rg-microproyecto2 --name
aks-microproyecto2
kubectl get nodes
kubectl get nodes -o wide
```

Ambos nodos aparecieron en estado Ready con version v1.34.8, Ubuntu 22.04 LTS y containerd 1.7.31. Como segundo metodo, desde Azure CLI en Windows se autentico con az login --use-device-code (necesario porque el correo UAO no esta registrado como cuenta Microsoft personal), se instalo kubectl con az aks install-cli y se verificaron los nodos obteniendo el mismo resultado.

4.2 Aplicacion de Clasificacion de Imagenes (Deep Learning)

Se desplegó un servidor Jupyter Notebook con TensorFlow usando los comandos:

```
kubectl create deployment image-classifier
--image=tensorflow/tensorflow:latest-jupyter
kubectl expose deployment image-classifier --type=LoadBalancer --port=8888
```

El servicio de tipo LoadBalancer aprovisiono automaticamente un balanceador de carga de Azure con IP publica 52.177.38.104. Se accedio al Jupyter desde el navegador y se implemento un notebook con MobileNetV2 preentrenado en ImageNet. Para evitar dependencia de internet se genero una imagen sintetica con NumPy (valores aleatorios 50-200, dimensiones 224x224x3) y se ejecuto la inferencia obteniendo: velvet 82.26%, wool 11.51%, lampshade 1.20%.

4.3 Aplicacion de Interes: Grafana

Se desplegó Grafana v13.0.2 con los siguientes comandos:

```
kubectl create deployment grafana --image=grafana/grafana:latest
kubectl expose deployment grafana --type=LoadBalancer --port=3000
```

La IP publica asignada fue 52.184.200.57. Se accedio al dashboard web, se cambio la contraseña por defecto y se exploro la interfaz con sus secciones de Dashboards, Connections, Alerting y Administration. Grafana es la herramienta estandar de monitoreo en entornos Kubernetes, tipicamente integrada con Prometheus para recopilar metricas del cluster.

4.4 Supervision y Monitoreo con Azure Monitor

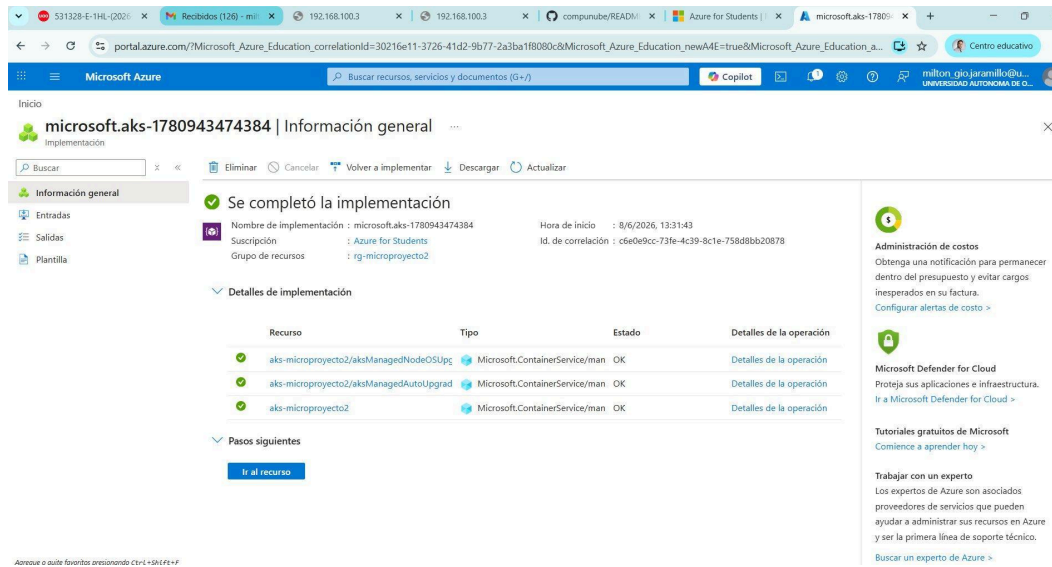
Desde la seccion "Supervisar" del cluster en Azure Portal se visualizo el estado en tiempo real: 2 nodos en estado Listo, 25 pods en ejecucion (0 errores), y 42 eventos totales (3

advertencias, 39 normales). Las metricas de infraestructura mostraron un ancho de banda de disco OS del 68% (vmss_1) y IOPS del 52%, valores elevados coincidentes con la descarga de imagenes Docker y pesos del modelo.

Desde "Supervision → Metricas" se configuraron graficas especificas: CPU Usage Percentage (Max) registro un pico del 47% durante la inferencia del modelo de deep learning, y API Server Memory Usage (Max) se mantuvo estable en el 15% a lo largo de 24 horas, indicando que el plano de control opera con amplio margen de capacidad.

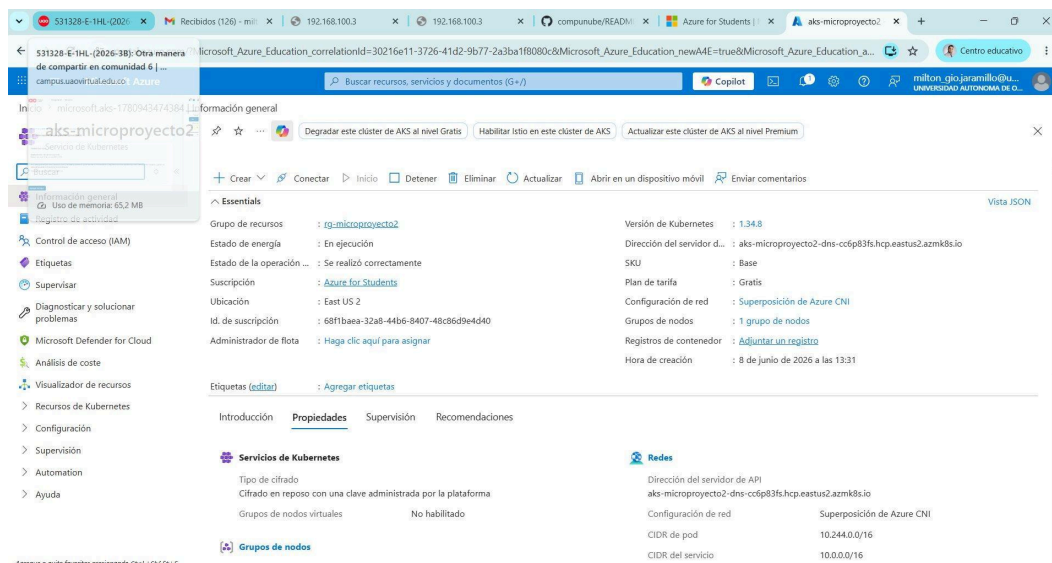
5. Capturas de Pantalla

Figura 1 - Implementacion del cluster aks-microproyecto2 completada en Azure Portal



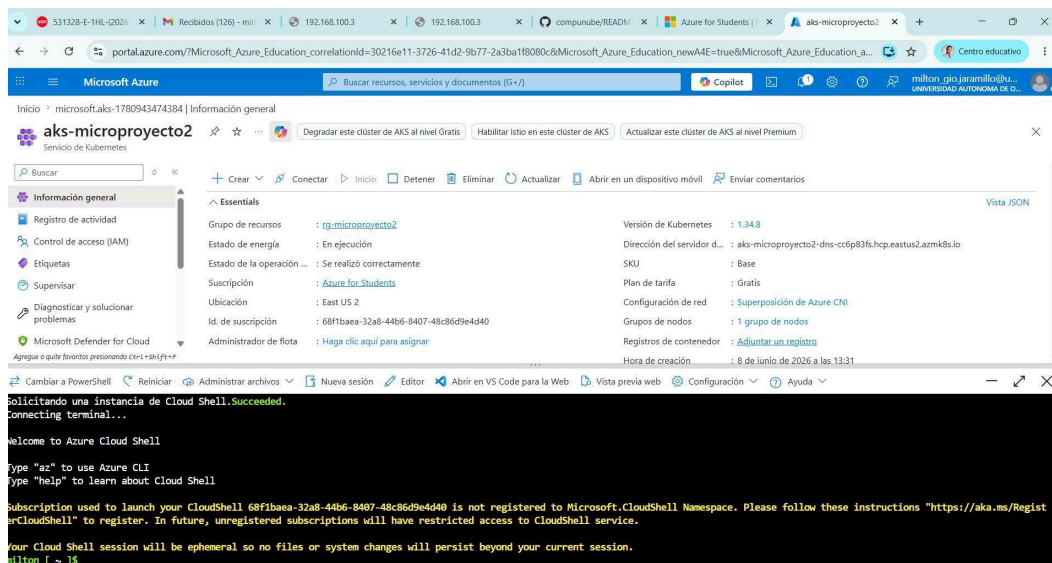
Implementacion completada: grupo de recursos rg-microproyecto2, suscripcion Azure for Students

Figura 2 - Información general del cluster: version 1.34.8, East US 2, Azure CNI, plan Gratis



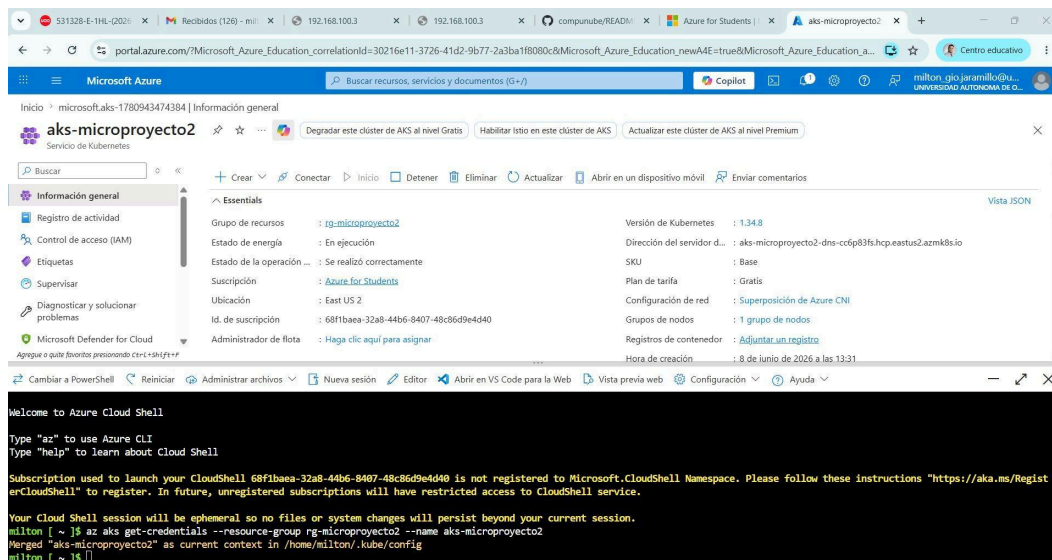
Cluster en ejecucion: Kubernetes 1.34.8, CIDR de pod 10.244.0.0/16, CIDR de servicio 10.0.0.0/16

Figura 3 - Azure Cloud Shell abierto con sesion activa



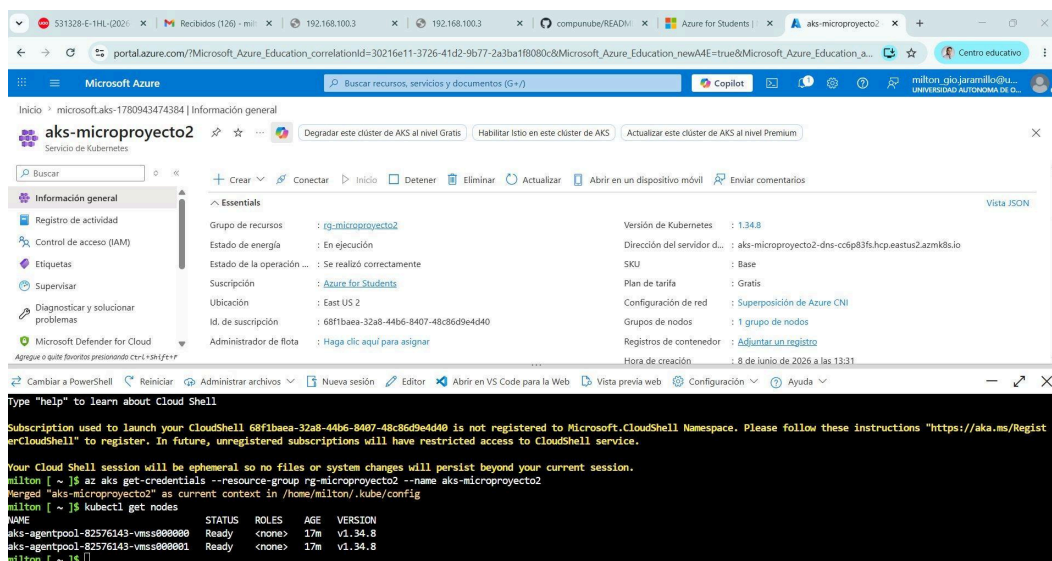
Cloud Shell bash iniciado desde el navegador con el cluster AKS en vista superior

Figura 4 - Comando az aks get-credentials: contexto del cluster configurado



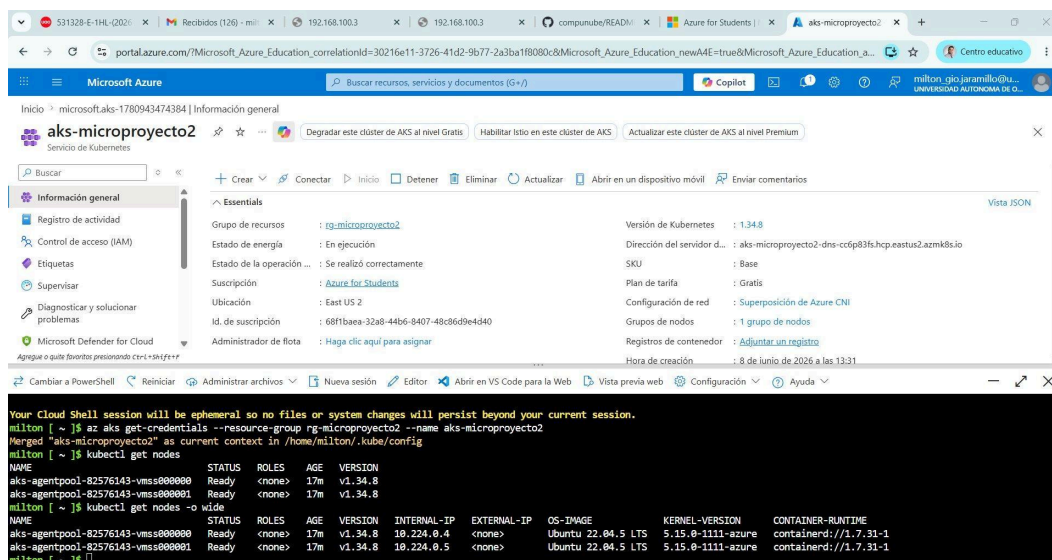
Credenciales descargadas: "Merged aks-microproyecto2 as current context"

Figura 5 - kubectl get nodes: ambos nodos en estado Ready v1.34.8



vmss000000 y vmss000001 en estado Ready con 17 minutos de funcionamiento

Figura 6 - kubectl get nodes -o wide: IPs internas, Ubuntu 22.04 LTS, containerd 1.7.31



IPs internas 10.224.0.4 y 10.224.0.5, OS Ubuntu 22.04 LTS, runtime containerd://1.7.31-1

Figura 7 - Azure CLI v2.87.0 instalado en Windows

```
Símbolo del sistema
Microsoft Windows [Versión 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\USUARIO>az --version
azure-cli                2.87.0
core                     2.87.0
telemetry                1.1.0

Dependencies:
msal                     1.36.0
azure-mgmt-resource      24.0.0

Python location 'C:\Program Files (x86)\Microsoft SDKs\Azure\CLI2\python.exe'
Config directory 'C:\Users\USUARIO\.azure'
Extensions directory 'C:\Users\USUARIO\.azure\cliextensions'

Python (Windows) 3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:11:29) [MSC v.1944 32 bit (Intel)]

Legal docs and information: aka.ms/AzureCliLegal

Your CLI is up-to-date.

C:\Users\USUARIO>
```

Salida de az --version: azure-cli 2.87.0, Python 3.13.13 en Windows

Figura 8 - kubectl get nodes desde Windows CMD: nodos Ready tras 46 minutos

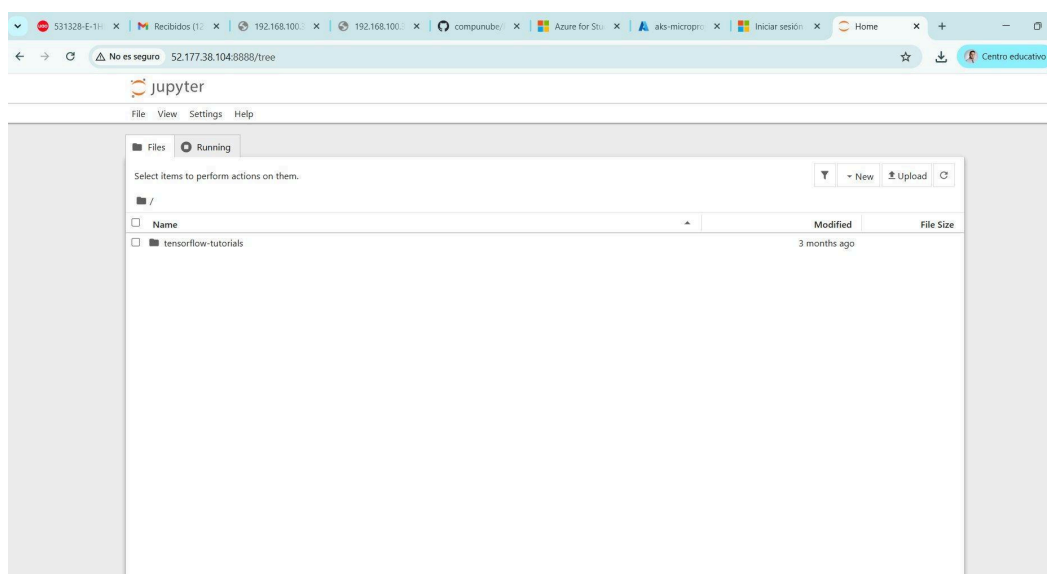
```
Símbolo del sistema
Microsoft Windows [Versión 10.0.26200.8457]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\USUARIO>kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
aks-agentpool-82576143-vmss000000 Ready    <none>   46m   v1.34.8
aks-agentpool-82576143-vmss000001 Ready    <none>   46m   v1.34.8

C:\Users\USUARIO>
```

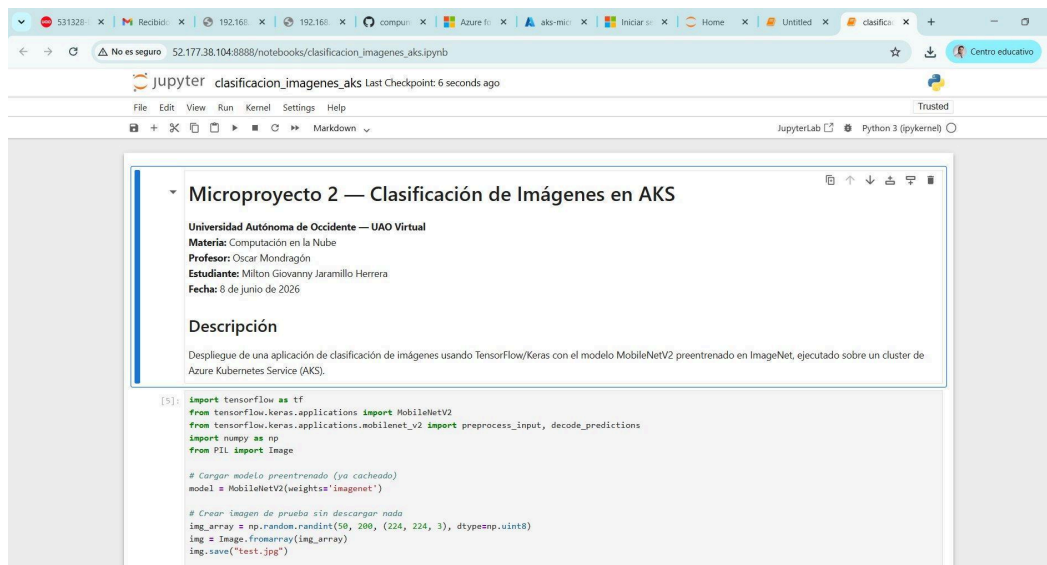
Verificación exitosa del cluster desde la terminal de Windows con kubectl configurado localmente

Figura 9 - Jupyter Notebook accesible en 52.177.38.104:8888



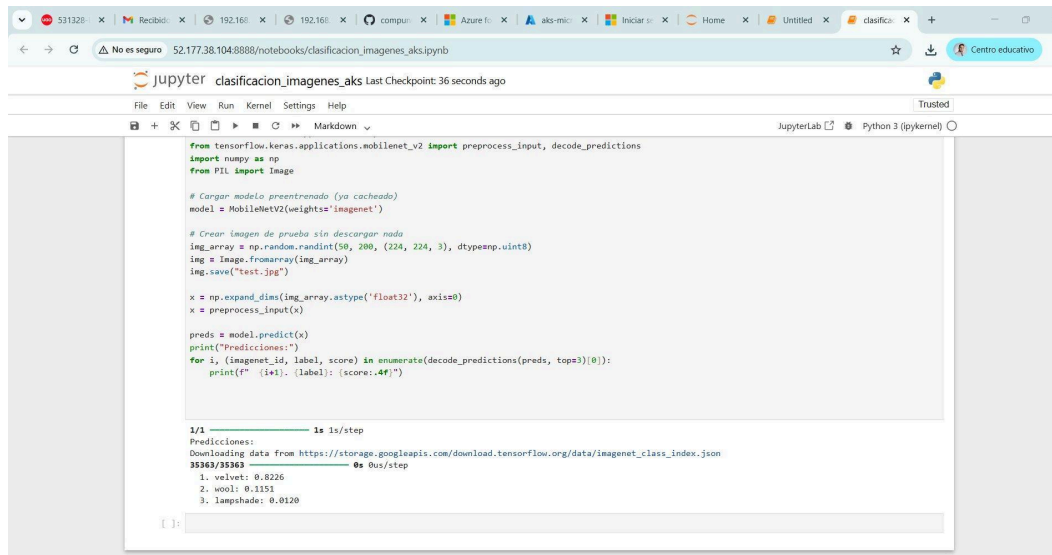
Interfaz de Jupyter con carpeta tensorflow-tutorials disponible en el pod

Figura 10a - Notebook de clasificacion: codigo MobileNetV2 con imagen sintetica NumPy



Notebook clasificacion_imagenes_aks.ipynb: carga del modelo y generacion de imagen de prueba

Figura 10b - Resultado de inferencia: velvet 82.26%, wool 11.51%, lampshade 1.20%



```
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions
import numpy as np
from PIL import Image

# Cargar modelo preentrenado (ya cachado)
model = MobileNetV2(weights='imagenet')

# Crear imagen de prueba sin descargar nada
img_array = np.random.randint(50, 200, (224, 224, 3), dtype=np.uint8)
img = Image.fromarray(img_array)
img.save("test.jpg")

x = np.expand_dims(img_array.astype('float32'), axis=0)
x = preprocess_input(x)

preds = model.predict(x)
print("Predicciones:")
for i, (imagenet_id, label, score) in enumerate(decode_predictions(preds, top3)[0]):
    print(f"{i+1}. {imagenet_id}. {label}: {score:.4f}")
```

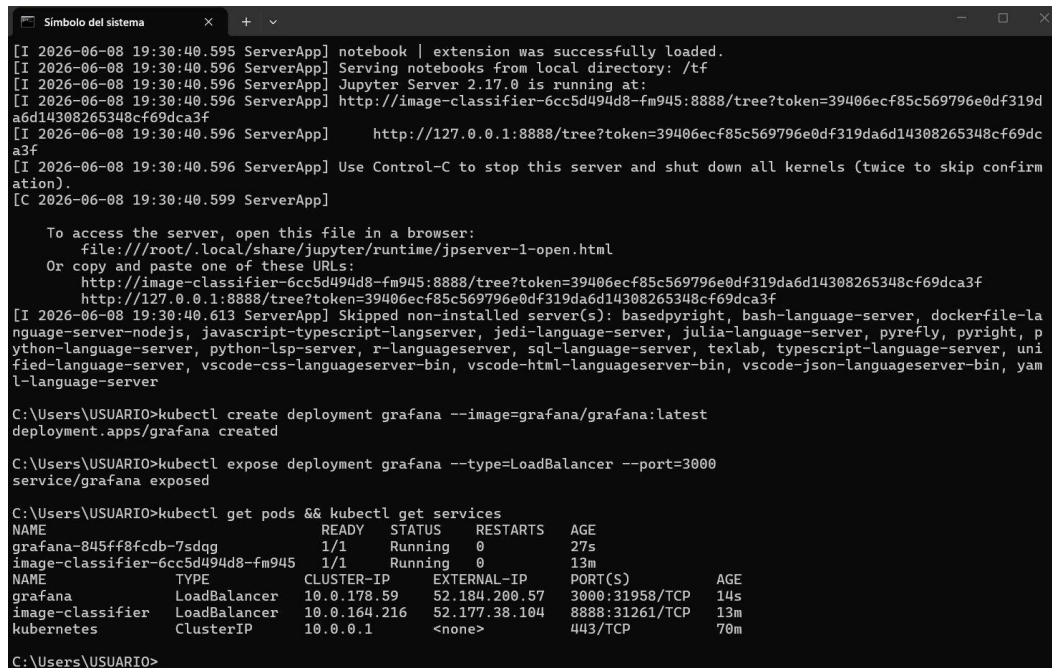
1/1 ————— 1s 1s/step

Predicciones:
Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_index.json
35863/35863 ————— 0s 0s/step

1. velvet: 0.8226
2. wool: 0.1151
3. lampshade: 0.0120

Predicciones top-3 del modelo MobileNetV2 sobre imagen sintetica generada con NumPy

Figura 11 - Grafana: pods y servicios activos con IPs publicas asignadas



```
[I 2026-06-08 19:30:40.595 ServerApp] notebook | extension was successfully loaded.
[I 2026-06-08 19:30:40.596 ServerApp] Serving notebooks from local directory: /tf
[I 2026-06-08 19:30:40.596 ServerApp] Jupyter Server 2.17.0 is running at:
[I 2026-06-08 19:30:40.596 ServerApp] http://image-classifier-6cc5d494d8-fm945:8888/tree?token=39406ecf85c569796e0df319da6d14308265348cf69dca3f
[I 2026-06-08 19:30:40.596 ServerApp] http://127.0.0.1:8888/tree?token=39406ecf85c569796e0df319da6d14308265348cf69dca3f
[I 2026-06-08 19:30:40.596 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 2026-06-08 19:30:40.599 ServerApp]

To access the server, open this file in a browser:
file:///root/.local/share/jupyter/runtime/jpserver-1-open.html
Or copy and paste one of these URLs:
http://image-classifier-6cc5d494d8-fm945:8888/tree?token=39406ecf85c569796e0df319da6d14308265348cf69dca3f
http://127.0.0.1:8888/tree?token=39406ecf85c569796e0df319da6d14308265348cf69dca3f
[I 2026-06-08 19:30:40.613 ServerApp] Skipped non-installed server(s): basedpyright, bash-language-server, dockerfile-language-server-nodejs, javascript-typescript-languageserver, jedi-language-server, julia-language-server, pyrefly, pyright, python-language-server, python-lsp-server, r-languageserver, sql-language-server, texlab, typescript-language-server, unified-language-server, vscode-css-languageserver-bin, vscode-html-languageserver-bin, vscode-json-languageserver-bin, yamll-language-server

C:\Users\USUARIO>kubectl create deployment grafana --image=grafana/grafana:latest
deployment.apps/grafana created

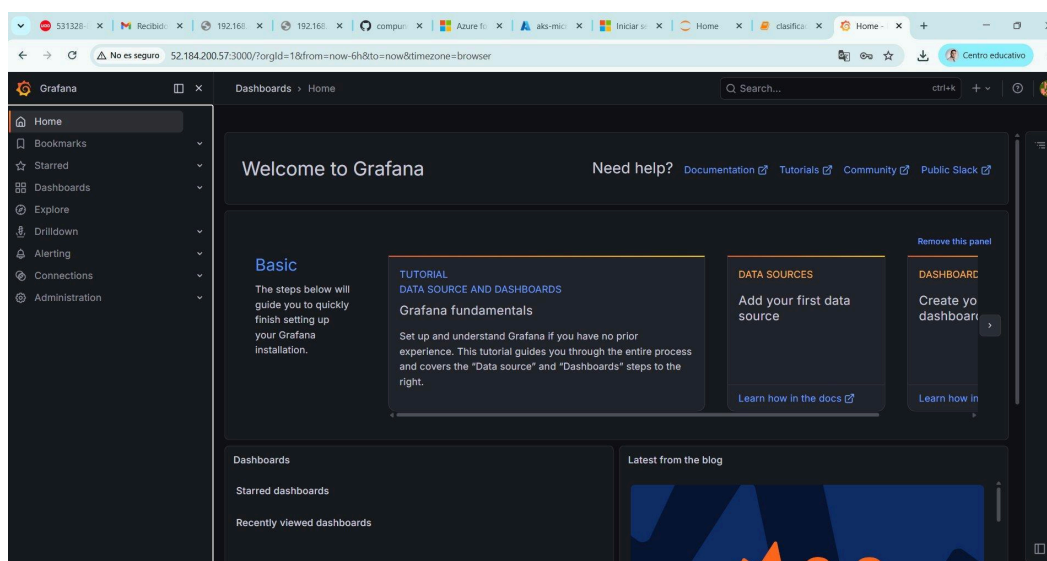
C:\Users\USUARIO>kubectl expose deployment grafana --type=LoadBalancer --port=3000
service/grafana exposed

C:\Users\USUARIO>kubectl get pods && kubectl get services
NAME                                READY   STATUS    RESTARTS   AGE
grafana-845ff8fdb-7sdqg             1/1     Running   0           27s
image-classifier-6cc5d494d8-fm945   1/1     Running   0           13m
NAME                                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
grafana                             LoadBalancer 10.0.178.59   52.184.200.57 3000:31958/TCP   14s
image-classifier                    LoadBalancer 10.0.164.216 52.177.38.104 8888:31261/TCP   13m
kubernetes                          ClusterIP     10.0.0.1     <none>         443/TCP          70m

C:\Users\USUARIO>
```

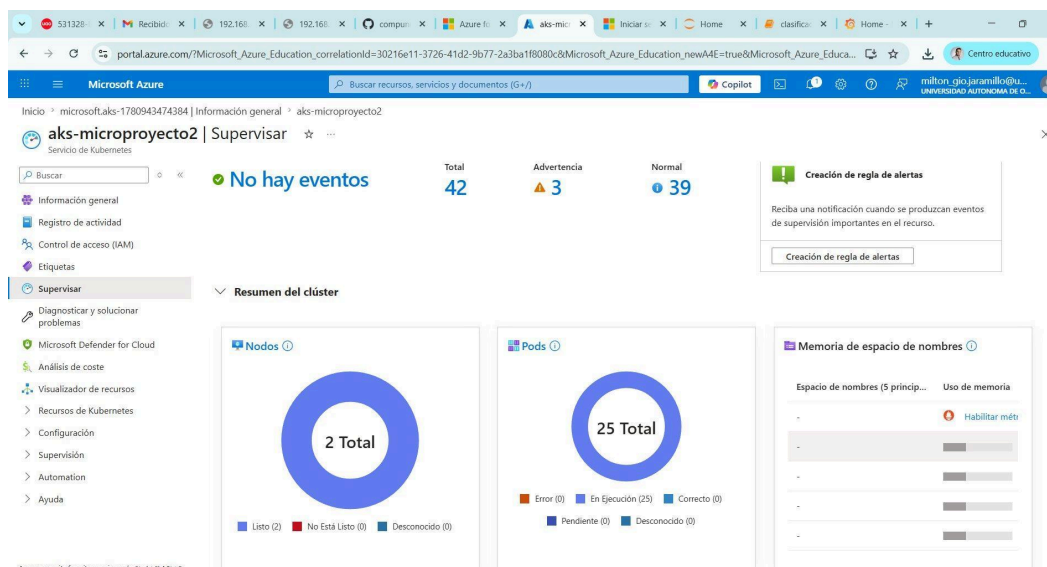
kubectl get pods && kubectl get services: grafana (52.184.200.57) e image-classifier (52.177.38.104) en Running

Figura 12 - Dashboard principal de Grafana v13.0.2 en 52.184.200.57:3000



Interfaz de Grafana con menu lateral: Home, Dashboards, Connections, Alerting, Administration

Figura 13a - Panel de supervision AKS: 2 nodos listos, 25 pods en ejecucion, 42 eventos



Azure Monitor: resumen del cluster con 2 nodos Listo, 25 pods en Ejecucion (0 errores)

Figura 13b - Metricas de infraestructura: ancho de banda disco OS 68%, IOPS 52%

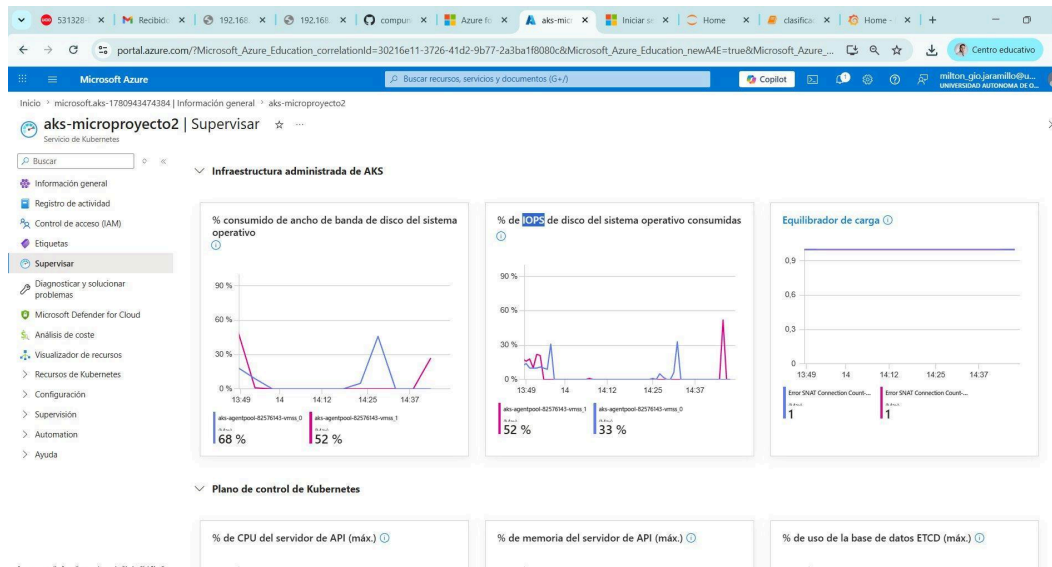


Figura 14 - API Server Memory Usage (Max): 15% estable en 24 horas

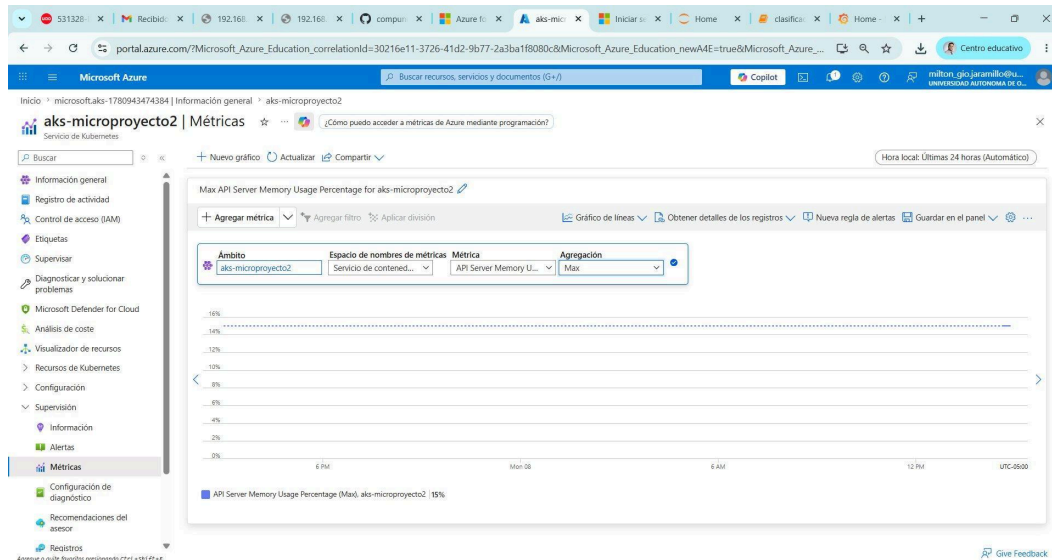
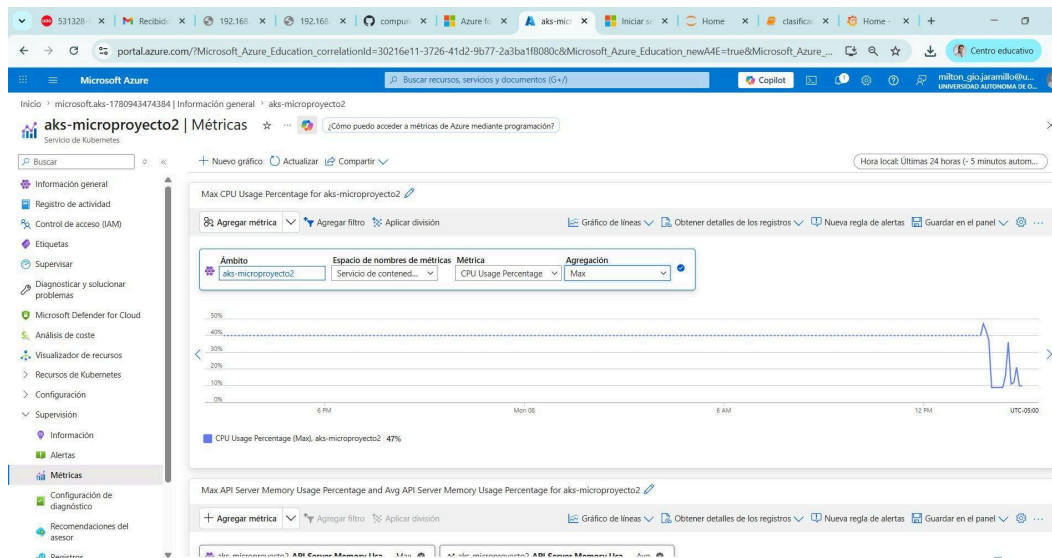


Figura 15 - CPU Usage Percentage (Max): pico de 47% durante inferencia del modelo

CPU maximo del cluster: 47% durante la ejecucion del modelo MobileNetV2 en el nodo AKS

6. Conclusiones

Este microproyecto permitio entender de manera practica el funcionamiento de Kubernetes como plataforma de orquestacion de contenedores en la nube. Con solo dos comandos (kubectl create deployment y kubectl expose) fue posible desplegar aplicaciones complejas como un servidor Jupyter con TensorFlow o Grafana, y obtener una IP publica funcional sin configurar manualmente balanceadores de carga ni reglas de firewall. Este nivel de automatizacion contrasta notablemente con la gestion manual de maquinas virtuales mediante Vagrant, donde cada aspecto de red e infraestructura debe especificarse explicitamente.

Comparando con tecnologias de modulos anteriores: Vagrant provee entornos reproducibles pero estaticos y orientados al desarrollo local; LXD ofrece contenedores ligeros pero requiere gestion manual del cluster; HAProxy permite balanceo de carga pero exige configuracion por servicio. Kubernetes unifica estas capacidades en una plataforma declarativa, y AKS lleva la abstraccion un nivel mas alla al gestionar el plano de control como servicio. El despliegue de MobileNetV2 demostro ademas que AKS es capaz de ejecutar cargas de trabajo de inteligencia artificial sin modificaciones especiales, sentando una base solida para proyectos mas avanzados del curso.

7. Referencias

- Microsoft. (2026). Azure Kubernetes Service (AKS) documentation. <https://learn.microsoft.com/en-us/azure/aks/>
- Kubernetes.io. (2026). Kubernetes Documentation. <https://kubernetes.io/docs/home/>
- TensorFlow. (2026). TensorFlow Documentation. https://www.tensorflow.org/api_docs
- Grafana Labs. (2026). Grafana Documentation. <https://grafana.com/docs/grafana/latest/>
- Microsoft. (2026). Azure Monitor — AKS Monitoring. <https://learn.microsoft.com/en-us/azure/aks/monitor-aks>
- Sandler, M. et al. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. arXiv:1801.04381.
- Martinez Velasquez, H. F. (2026). Guia Microproyecto 2 — Kubernetes en Azure AKS. UAO Virtual.